

ECE 522/622 - Spring 2026

Modeling and Verification of Embedded Systems

Lab #4: Open Ended

The objective of Lab 4 is to apply knowledge from the course, and to practice working without explicit step-by-step instructions. Choose a project that is interesting but can reasonably be completed in ~10 hours of work over three weeks. A project can have some risk; negative results can earn a high score if the work quality is good. Below are example project topics. You can pursue one of these, or your own idea. Grading will consider: (1) problem formulation, (2) correctness of modeling, (3) depth of analysis/insight, and (4) clarity of report/results. Bonus for originality/difficulty.

General instructions:

Lab 4 is to be completed individually. Lab 4 is required for ECE622 students, but optional for ECE522 students.

- ECE522 students that opt in to lab 4 should email me by 4/30 to declare.
- ECE522 students that skip Lab 4 will have their Lab 4 score replaced by their Exam 1 score.

We will have a **project check-in meeting on 5/5/26** instead of lecture. Each person with a project will briefly explain:

1. What their project is about, and how it relates to topics from class
2. What tool(s) the project uses
3. What are the data/results they intend to show at completion
4. What is the main risk/uncertainty

Reports are due on 5/12/26. Lab reports should be typed (~2-4 pages), with name and ID at the beginning. Upload all source files with your submission. We will add post-submission meetings as needed to review files and report.

AI Usage Guidelines:

Effective AI use is a skill. If you use AI tools, you must submit the full source showing your usage, as an attachment. For example, if using an LLM, provide a pdf of the full unedited chat log. Projects using AI will be evaluated on the quality of the AI usage. Consider the following indicators of good vs bad usage. If you have questions about AI usage, just ask.

- **Good AI usage:** You are in the loop at each step, learning; you have deep understanding of your project at the end; AI tools help you by generating code, debugging, explaining concepts, generating ideas.
- **Bad AI usage:** You ask the AI to generate the idea, code, and report; you are not learning; you don't have a deep understanding of your project.

Example Project Ideas:

Topics related to tools and capabilities:

- Model a simple system using the **PRISM** probabilistic model checker. Do probabilities of "0" and ">0" in PRISM correspond to "unreachable" and "reachable" in a non-probabilistic reachability check as in Lab 1?
- Create models in the **Promela** language and then use the **SPIN** model checker to check reachability properties. Compare/contrast against Lab 1 approach.
- Try same as above but using the **nuXmv** model checker instead of SPIN.
- Apply some advanced features of UPPAAL or PHAVerLite (e.g. add forward-backward refinement in PHAVerLite).
- Use tools/algorithms to automatically generate (Büchi) automata for checking LTL formulas.

Topics related to modeling of systems:

- An SRAM cell has 2 state nodes and can be represented in PHAVerLite as a system with 2 variables. Try to prove, for some range of initial conditions, that the cell will not spontaneously flip its stored bit. In other words, that it cannot reach the part of the state space that corresponds to the opposite stored bit.
- A UART receiver circuit synchronizes on the falling edge of a data line and then counts clock cycles to determine when to sample the subsequent bits on the data line. Sampling errors can occur if the receiver clock is much faster or slower than it should be. Model this in UPPAAL or PHAVerLite and perform analysis to find the smallest amount of clock period error that can cause bits to be sampled incorrectly.
- Verify path delay bounds on a simple combinational circuit path using timed automata. Circuit path should have only a few gates. Check literature for prior approaches of using timed automata for circuit timing.
- Modify your lab 1 code to create an inductive proof that certain states are unreachable. Induction can prove unreachability in a single step by showing that a state is unreachable from non-deterministic initial state.
- Use a hybrid systems verifier like PHAVerLite to model kinetic time dilation from special relativity and then analyze the twin paradox to show that twins can reach different ages due to their motion.